STA6166 Statistical Methods I — Final Project R Code

This document contains **only Questions 1–4, 6, 10, and 11**, corresponding to the supplied script. Question 6 demonstrates 12 alternatives; the project requires students to select only one method and one criterion.

Setup

Install packages once in the R console if needed: `install.packages(c("lmtest", "leaps", "glmnet"))`. Installation is not repeated during knitting.

```
setwd("C:/Users/n01654025/Downloads/STA6166/Final_project")
# Import the tab-delimited data as a data frame
data <- read.delim("STA6166_final_project_data.tsv", header = TRUE,
sep = "\t")
library(lmtest)
```

```
## Loading required package: zoo
```

```
##
## Attaching package: 'zoo'
```

```
## The following objects are masked from 'package:base':
##
##     as.Date, as.Date.numeric
```

```
library(leaps)
library(glmnet)
```

```
## Loading required package: Matrix
```

```
## Loaded glmnet 4.1-8
```

Question 1: Full Multiple Linear Regression

Fit all 15 predictors simultaneously. `summary()` displays the coefficient estimates, standard errors, t-statistics, p-values, and R-squared statistics.

```
# Fit the full regression model
full_model <- lm(Y ~ ., data = data)
# Display regression results
summary(full_model)
```

```
##
## Call:
## lm(formula = Y ~ ., data = data)
##
```

```
## Residuals:
##      Min       1Q   Median       3Q      Max
## -16.8549  -4.2110   0.7026   4.3100  14.9088
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)   1.5842     0.7122   2.224 0.028813 *
## X1            1.9115     0.7175   2.664 0.009253 **
## X2            0.2025     0.6982   0.290 0.772483
## X3           -0.3949     0.6771  -0.583 0.561335
## X4           -2.6473     0.8055  -3.287 0.001481 **
## X5            1.1445     0.7498   1.526 0.130681
## X6            0.9065     0.7347   1.234 0.220698
## X7            1.2767     0.6619   1.929 0.057121 .
## X8            0.2487     0.7648   0.325 0.745818
## X9           -0.1041     0.6638  -0.157 0.875731
## X10           7.2157     0.7176  10.056 4.49e-16 ***
## X11          -0.1946     0.6258  -0.311 0.756631
## X12           0.3334     0.6508   0.512 0.609722
## X13          -2.6675     0.6812  -3.916 0.000183 ***
## X14          -1.6059     0.6871  -2.337 0.021801 *
## X15           0.5023     0.6974   0.720 0.473400
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 6.427 on 84 degrees of freedom
## Multiple R-squared:  0.6506, Adjusted R-squared:  0.5882
## F-statistic: 10.43 on 15 and 84 DF,  p-value: 1.442e-13
```

Question 2: Breusch–Pagan Test

This test assesses constant conditional error variance. The null is homoscedasticity; a small p-value suggests heteroscedasticity. Extract the statistic, p-value, and degrees of freedom.

```
# Refit the full model, as in the original script
full_model <- lm(Y ~ ., data = data)
# Perform the Breusch-Pagan test
bp_result <- bptest(full_model)
print(bp_result)

##
##  studentized Breusch-Pagan test
##
## data:  full_model
## BP = 17.673, df = 15, p-value = 0.2802

# Extract test statistic, p-value, and degrees of freedom
bp_result$statistic
```

```
##       BP
## 17.67304

bp_result$p.value

##        BP
## 0.2802448

bp_result$parameter

## df
## 15
```

Question 3: Shapiro–Wilk Normality Diagnostic

Apply Shapiro–Wilk to the OLS residuals. This is a descriptive diagnostic: residuals are generally dependent and have unequal variances even when the underlying regression errors are normal, so the standard p-value is not an exact test of error normality.

```
full_model <- lm(Y ~ ., data = data)
# Extract OLS residuals
res <- residuals(full_model)
# Perform Shapiro-Wilk test
shapiro_result <- shapiro.test(res)
print(shapiro_result)

##
##  Shapiro-Wilk normality test
##
## data:  res
## W = 0.99144, p-value = 0.7795

# Extract test statistic and p-value
shapiro_result$statistic

##         W
## 0.9914416

shapiro_result$p.value

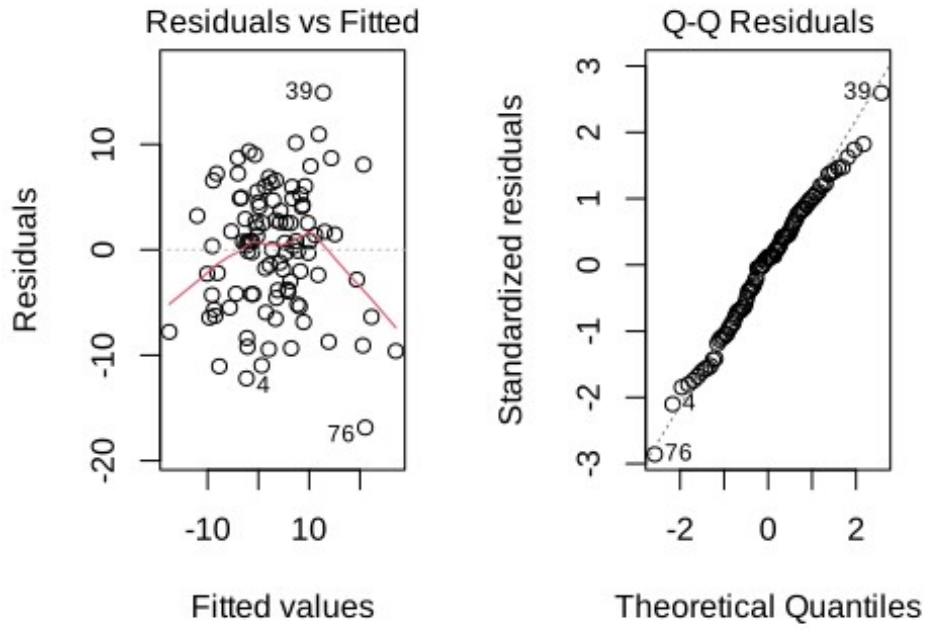
## [1] 0.7794585
```

Question 4: Regression Diagnostic Plots

The residuals-versus-fitted plot helps assess constant variance and systematic patterns. The Q–Q plot helps assess departures from normality.

```
full_model <- lm(Y ~ ., data = data)
# Display two plots side by side
par(mfrow = c(1, 2))
```

```r
# (a) Residuals versus fitted values
plot(full_model, which = 1)
# (b) Normal Q-Q plot
plot(full_model, which = 2)
```




```r
# Restore plotting layout
par(mfrow = c(1, 1))
```

Question 6: Classical Model Selection

AIC, BIC, and Mallows' C_p are **minimized**; adjusted R^2 is **maximized**. The `step()` function supports all three directions for AIC/BIC. `regsubsets()` supports forward and backward searches for C_p and adjusted R^2. A custom greedy bidirectional function is defined in 6.9 and reused in 6.12.

Shared setup

```r
full_model <- lm(Y ~ ., data = data)
null_model <- lm(Y ~ 1, data = data)
N <- nrow(data)
```

6.1 Forward — AIC

Start with the intercept-only model and add variables. `k = 2` applies the AIC penalty. Report the selected formula and AIC value.

```r
forward_AIC <- step(null_model,
  scope = list(lower = formula(null_model), upper =
formula(full_model)),
  direction = "forward",
  k = 2, trace = 0)
summary(forward_AIC)
```

```
##
## Call:
## lm(formula = Y ~ X10 + X4 + X13 + X14 + X1 + X7 + X5, data = data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -16.9862  -4.5421   0.7744   4.0540  15.0699
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)   1.4963     0.6695   2.235 0.027826 *
## X10           7.1034     0.6599  10.765  < 2e-16 ***
## X4           -2.5995     0.7703  -3.375 0.001084 **
## X13          -2.5217     0.6513  -3.872 0.000202 ***
## X14          -1.5138     0.6565  -2.306 0.023376 *
## X1            1.7693     0.6780   2.610 0.010575 *
## X7            1.2826     0.6155   2.084 0.039950 *
## X5            1.1386     0.7151   1.592 0.114777
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 6.24 on 92 degrees of freedom
## Multiple R-squared:  0.6392, Adjusted R-squared:  0.6118
## F-statistic: 23.29 on 7 and 92 DF,  p-value: < 2.2e-16
```

```r
formula(forward_AIC)
```

```
## Y ~ X10 + X4 + X13 + X14 + X1 + X7 + X5
```

```r
AIC(forward_AIC)
```

```
## [1] 659.6584
```

6.2 Backward — AIC

Start with all variables and remove variables. `k = 2` applies the AIC penalty. Report the selected formula and AIC value.

```r
backward_AIC <- step(full_model, direction = "backward", k = 2, trace
= 0)
summary(backward_AIC)
```

```
##
## Call:
## lm(formula = Y ~ X1 + X4 + X5 + X7 + X10 + X13 + X14, data = data)
```

```
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -16.9862  -4.5421   0.7744   4.0540  15.0699
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)   1.4963     0.6695   2.235 0.027826 *
## X1            1.7693     0.6780   2.610 0.010575 *
## X4           -2.5995     0.7703  -3.375 0.001084 **
## X5            1.1386     0.7151   1.592 0.114777
## X7            1.2826     0.6155   2.084 0.039950 *
## X10           7.1034     0.6599  10.765  < 2e-16 ***
## X13          -2.5217     0.6513  -3.872 0.000202 ***
## X14          -1.5138     0.6565  -2.306 0.023376 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 6.24 on 92 degrees of freedom
## Multiple R-squared:  0.6392, Adjusted R-squared:  0.6118
## F-statistic: 23.29 on 7 and 92 DF,  p-value: < 2.2e-16
```

```
formula(backward_AIC)
```

```
## Y ~ X1 + X4 + X5 + X7 + X10 + X13 + X14
```

```
AIC(backward_AIC)
```

```
## [1] 659.6584
```

6.3 Bidirectional stepwise — AIC

Start with the intercept-only model and allow additions or removals. `k = 2` applies the AIC penalty. Report the selected formula and AIC value.

```
stepwise_AIC <- step(null_model,
  scope = list(lower = formula(null_model), upper =
formula(full_model)),
  direction = "both",
  k = 2, trace = 0)
summary(stepwise_AIC)
```

```
##
## Call:
## lm(formula = Y ~ X10 + X4 + X13 + X14 + X1 + X7 + X5, data = data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -16.9862  -4.5421   0.7744   4.0540  15.0699
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
```

```
## (Intercept)   1.4963     0.6695   2.235 0.027826 *
## X10           7.1034     0.6599  10.765  < 2e-16 ***
## X4           -2.5995     0.7703  -3.375 0.001084 **
## X13          -2.5217     0.6513  -3.872 0.000202 ***
## X14          -1.5138     0.6565  -2.306 0.023376 *
## X1            1.7693     0.6780   2.610 0.010575 *
## X7            1.2826     0.6155   2.084 0.039950 *
## X5            1.1386     0.7151   1.592 0.114777
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 6.24 on 92 degrees of freedom
## Multiple R-squared:  0.6392, Adjusted R-squared:  0.6118
## F-statistic: 23.29 on 7 and 92 DF,  p-value: < 2.2e-16
```

```
formula(stepwise_AIC)
```

```
## Y ~ X10 + X4 + X13 + X14 + X1 + X7 + X5
```

```
AIC(stepwise_AIC)
```

```
## [1] 659.6584
```

6.4 Forward — BIC

Start with the intercept-only model and add variables. `k = log(N)` applies the BIC penalty. Report the selected formula and BIC value.

```
forward_BIC <- step(null_model,
  scope = list(lower = formula(null_model), upper =
formula(full_model)),
  direction = "forward",
  k = log(N), trace = 0)
summary(forward_BIC)
```

```
##
## Call:
## lm(formula = Y ~ X10 + X4 + X13 + X14 + X1, data = data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -17.3617  -4.4257  -0.0181   4.1256  15.1591
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)   1.9307     0.6628   2.913 0.004473 **
## X10           7.2586     0.6732  10.782  < 2e-16 ***
## X4           -2.6507     0.7873  -3.367 0.001103 **
## X13          -2.3389     0.6343  -3.687 0.000379 ***
## X14          -1.8656     0.6545  -2.850 0.005366 **
## X1            1.6604     0.6892   2.409 0.017936 *
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 6.393 on 94 degrees of freedom
## Multiple R-squared:  0.6131, Adjusted R-squared:  0.5925
## F-statistic: 29.79 on 5 and 94 DF,  p-value: < 2.2e-16
```

```
formula(forward_BIC)
```

```
## Y ~ X10 + X4 + X13 + X14 + X1
```

```
BIC(forward_BIC)
```

```
## [1] 680.8865
```

6.5 Backward — BIC

Start with all variables and remove variables. `k = log(N)` applies the BIC penalty. Report the selected formula and BIC value.

```
backward_BIC <- step(full_model, direction = "backward", k = log(N),
trace = 0)
summary(backward_BIC)
```

```
##
## Call:
## lm(formula = Y ~ X1 + X4 + X10 + X13 + X14, data = data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -17.3617  -4.4257  -0.0181   4.1256  15.1591
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)   1.9307     0.6628   2.913 0.004473 **
## X1            1.6604     0.6892   2.409 0.017936 *
## X4           -2.6507     0.7873  -3.367 0.001103 **
## X10           7.2586     0.6732  10.782  < 2e-16 ***
## X13          -2.3389     0.6343  -3.687 0.000379 ***
## X14          -1.8656     0.6545  -2.850 0.005366 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 6.393 on 94 degrees of freedom
## Multiple R-squared:  0.6131, Adjusted R-squared:  0.5925
## F-statistic: 29.79 on 5 and 94 DF,  p-value: < 2.2e-16
```

```
formula(backward_BIC)
```

```
## Y ~ X1 + X4 + X10 + X13 + X14
```

```
BIC(backward_BIC)
```

```
## [1] 680.8865
```

6.6 Bidirectional stepwise — BIC

Start with the intercept-only model and allow additions or removals. `k = log(N)` applies the BIC penalty. Report the selected formula and BIC value.

```
stepwise_BIC <- step(null_model,
  scope = list(lower = formula(null_model), upper =
formula(full_model)),
  direction = "both",
  k = log(N), trace = 0)
summary(stepwise_BIC)

##
## Call:
## lm(formula = Y ~ X10 + X4 + X13 + X14 + X1, data = data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -17.3617  -4.4257  -0.0181   4.1256  15.1591
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)   1.9307     0.6628   2.913 0.004473 **
## X10           7.2586     0.6732  10.782  < 2e-16 ***
## X4           -2.6507     0.7873  -3.367 0.001103 **
## X13          -2.3389     0.6343  -3.687 0.000379 ***
## X14          -1.8656     0.6545  -2.850 0.005366 **
## X1            1.6604     0.6892   2.409 0.017936 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 6.393 on 94 degrees of freedom
## Multiple R-squared:  0.6131, Adjusted R-squared:  0.5925
## F-statistic: 29.79 on 5 and 94 DF,  p-value: < 2.2e-16

formula(stepwise_BIC)

## Y ~ X10 + X4 + X13 + X14 + X1

BIC(stepwise_BIC)

## [1] 680.8865
```

6.7 Forward — Mallows' C_p

`regsubsets()` builds a forward search path. `which.min()` finds the model size with minimum C_p along that path.

```
forward_Cp <- regsubsets(Y ~ ., data = data, nvmax = 15, method =
"forward")
```

```r
results <- summary(forward_Cp)
# Select the model with minimum Cp
best_size <- which.min(results$cp)
# Selected coefficients, Cp, and predictor names
coef(forward_Cp, id = best_size)
```

```
## (Intercept)          X1          X4          X5          X7
X10
##    1.496340    1.769348   -2.599492    1.138639    1.282561
7.103359
##         X13         X14
##   -2.521701   -1.513772
```

```r
results$cp[best_size]
```

```
## [1] 2.7414
```

```r
names(coef(forward_Cp, id = best_size))[-1]
```

```
## [1] "X1"  "X4"  "X5"  "X7"  "X10" "X13" "X14"
```

6.8 Backward — Mallows' C_p

`regsubsets()` builds a backward search path. `which.min()` finds the model size with minimum C_p along that path.

```r
backward_Cp <- regsubsets(Y ~ ., data = data, nvmax = 15, method =
"backward")
results <- summary(backward_Cp)
# Select the model with minimum Cp
best_size <- which.min(results$cp)
# Selected coefficients, Cp, and predictor names
coef(backward_Cp, id = best_size)
```

```
## (Intercept)          X1          X4          X5          X7
X10
##    1.496340    1.769348   -2.599492    1.138639    1.282561
7.103359
##         X13         X14
##   -2.521701   -1.513772
```

```r
results$cp[best_size]
```

```
## [1] 2.7414
```

```r
names(coef(backward_Cp, id = best_size))[-1]
```

```
## [1] "X1"  "X4"  "X5"  "X7"  "X10" "X13" "X14"
```

6.9 Bidirectional stepwise — Mallows' C_p

The following original custom function evaluates one-predictor additions and removals, moving to the best improving neighbor. It stops at a local optimum and does not guarantee the globally best subset. For C_p, the full-model residual variance is used as the common reference.

```
stepwise_custom <- function(data, criterion = "Cp") {
  predictors <- setdiff(names(data), "Y")
  N <- nrow(data)
  # Full model for estimating residual variance
  full <- lm(Y ~ ., data = data)
  sigma2_full <- sum(resid(full)^2) / df.residual(full)
  TSS <- sum((data$Y - mean(data$Y))^2)
  # Score a candidate model
  evaluate <- function(vars) {
    # reformulate() needs at least one term; "1" gives the intercept-
only model
    f <- reformulate(if (length(vars) > 0) vars else "1", response =
"Y")
    fit <- lm(f, data = data)
    RSS <- sum(resid(fit)^2)
    p <- length(coef(fit)) # Includes intercept
    if (criterion == "Cp") {
      score <- RSS / sigma2_full - N + 2 * p
    } else if (criterion == "adjR2") {
      score <- 1 - (RSS / (N - p)) / (TSS / (N - 1))
    } else {
      stop("Criterion must be Cp or adjR2")
    }
    score
  }
  # Start with intercept-only model
  selected <- character(0)
  current_score <- evaluate(selected)
  repeat {
    remaining <- setdiff(predictors, selected)
    # Candidate additions
    add_models <- lapply(remaining, function(v) c(selected, v))
    # Candidate removals
    remove_models <- lapply(selected, function(v) setdiff(selected,
v))
    candidates <- c(add_models, remove_models)
    if (length(candidates) == 0) break
    scores <- vapply(candidates, evaluate, numeric(1))
    if (criterion == "Cp") {
      best <- which.min(scores)
      improved <- scores[best] < current_score - 1e-10
    } else {
      best <- which.max(scores)
      improved <- scores[best] > current_score + 1e-10
```

```
    }
    if (!improved) break
    selected <- candidates[[best]]
    current_score <- scores[best]
  }
  final_model <- lm(reformulate(selected, response = "Y"), data =
data)
  list(model = final_model, selected = selected,
       criterion_value = current_score)
}

stepwise_Cp <- stepwise_custom(data = data, criterion = "Cp")
summary(stepwise_Cp$model)

##
## Call:
## lm(formula = reformulate(selected, response = "Y"), data = data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -16.9862  -4.5421   0.7744   4.0540  15.0699
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)   1.4963     0.6695   2.235 0.027826 *
## X10           7.1034     0.6599  10.765  < 2e-16 ***
## X4           -2.5995     0.7703  -3.375 0.001084 **
## X13          -2.5217     0.6513  -3.872 0.000202 ***
## X14          -1.5138     0.6565  -2.306 0.023376 *
## X1            1.7693     0.6780   2.610 0.010575 *
## X7            1.2826     0.6155   2.084 0.039950 *
## X5            1.1386     0.7151   1.592 0.114777
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 6.24 on 92 degrees of freedom
## Multiple R-squared:  0.6392, Adjusted R-squared:  0.6118
## F-statistic: 23.29 on 7 and 92 DF,  p-value: < 2.2e-16

formula(stepwise_Cp$model)

## Y ~ X10 + X4 + X13 + X14 + X1 + X7 + X5
## <environment: 0x558dbf0de430>

# Selected predictors and final Cp
stepwise_Cp$selected

## [1] "X10" "X4"  "X13" "X14" "X1"  "X7"  "X5"

stepwise_Cp$criterion_value

## [1] 2.7414
```

6.10 Forward — Adjusted R^2

`regsubsets()` builds a forward search path. `which.max()` chooses the model with the largest adjusted R^2 along that path.

```
forward_adjR2 <- regsubsets(Y ~ ., data = data, nvmax = 15, method =
"forward")
results <- summary(forward_adjR2)
# Choose the largest adjusted R-squared
best_size <- which.max(results$adjr2)
coef(forward_adjR2, id = best_size)
```

```
## (Intercept)          X1          X4          X5          X6
X7
##   1.5409515   1.7470194  -2.5685383   1.1094958   0.7762993
1.2986014
##         X10         X13         X14
##   7.2256587  -2.5595331  -1.5944143
```

```
results$adjr2[best_size]
```

```
## [1] 0.6130085
```

```
names(coef(forward_adjR2, id = best_size))[-1]
```

```
## [1] "X1"  "X4"  "X5"  "X6"  "X7"  "X10" "X13" "X14"
```

6.11 Backward — Adjusted R^2

`regsubsets()` builds a backward search path. `which.max()` chooses the model with the largest adjusted R^2 along that path.

```
backward_adjR2 <- regsubsets(Y ~ ., data = data, nvmax = 15, method =
"backward")
results <- summary(backward_adjR2)
# Choose the largest adjusted R-squared
best_size <- which.max(results$adjr2)
coef(backward_adjR2, id = best_size)
```

```
## (Intercept)          X1          X4          X5          X6
X7
##   1.5409515   1.7470194  -2.5685383   1.1094958   0.7762993
1.2986014
##         X10         X13         X14
##   7.2256587  -2.5595331  -1.5944143
```

```
results$adjr2[best_size]
```

```
## [1] 0.6130085
```

```
names(coef(backward_adjR2, id = best_size))[-1]
```

```
## [1] "X1"  "X4"  "X5"  "X6"  "X7"  "X10" "X13" "X14"
```

6.12 Bidirectional stepwise — Adjusted R^2

Reuse `stepwise_custom()` from 6.9, now maximizing adjusted R^2.

```
stepwise_adjR2 <- stepwise_custom(data = data, criterion = "adjR2")
summary(stepwise_adjR2$model)
```

```
##
## Call:
## lm(formula = reformulate(selected, response = "Y"), data = data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -17.1167  -4.2954   0.5313   4.2399  14.6216
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)   1.5410     0.6695   2.302 0.023645 *
## X10           7.2257     0.6675  10.824  < 2e-16 ***
## X4           -2.5685     0.7696  -3.338 0.001225 **
## X13          -2.5595     0.6511  -3.931 0.000165 ***
## X14          -1.5944     0.6593  -2.418 0.017587 *
## X1            1.7470     0.6772   2.580 0.011485 *
## X7            1.2986     0.6147   2.113 0.037367 *
## X5            1.1095     0.7145   1.553 0.123921
## X6            0.7763     0.6827   1.137 0.258500
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 6.23 on 91 degrees of freedom
## Multiple R-squared:  0.6443, Adjusted R-squared:  0.613
## F-statistic:  20.6 on 8 and 91 DF,  p-value: < 2.2e-16
```

```
formula(stepwise_adjR2$model)
```

```
## Y ~ X10 + X4 + X13 + X14 + X1 + X7 + X5 + X6
## <environment: 0x558dc12b85b0>
```

```
# Selected predictors and final adjusted R-squared
stepwise_adjR2$selected
```

```
## [1] "X10" "X4"  "X13" "X14" "X1"  "X7"  "X5"  "X6"
```

```
stepwise_adjR2$criterion_value
```

```
## [1] 0.6130085
```

Question 10: Choose LASSO λ by Cross-Validation

`glmnet` uses a numeric predictor matrix. With `alpha = 1`, the model is LASSO; `nfolds = 10` specifies ten-fold cross-validation, and `type.measure = "mse"` evaluates mean squared error. The seed makes the fold allocation reproducible.

Step 1. Prepare the data

```
# Predictor matrix: X1 through X15
X <- as.matrix(data[, paste0("X", 1:15)])
# Response vector
Y <- data$Y
# Check dimensions
dim(X)

## [1] 100  15

length(Y)

## [1] 100
```

Step 2. Run 10-fold cross-validation

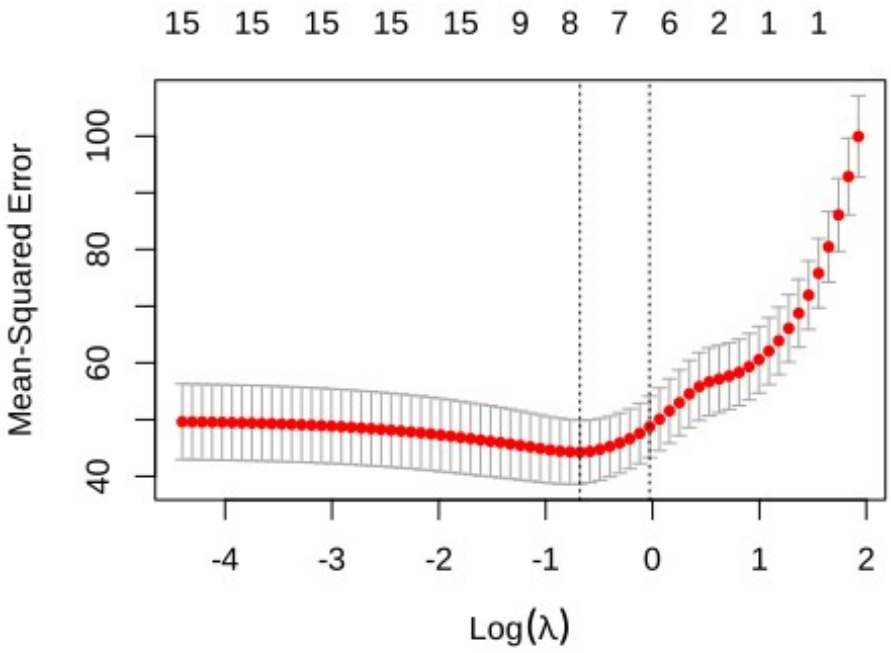
```
set.seed(6166)
cv_lasso <- cv.glmnet(x = X, y = Y, alpha = 1,
  nfolds = 10, family = "gaussian", type.measure = "mse",
  standardize = TRUE)
# Display cross-validation results
print(cv_lasso)

##
## Call:  cv.glmnet(x = X, y = Y, type.measure = "mse", nfolds = 10,
alpha = 1,      family = "gaussian", standardize = TRUE)
##
## Measure: Mean-Squared Error
##
##     Lambda Index Measure    SE Nonzero
## min 0.5070    29   44.27 5.628       8
## 1se 0.9725    22   48.73 5.419       6
```

Step 3. Plot cross-validation MSE

`plot()` shows the cross-validation error curve. A second call saves the plot to a PDF in the working directory.

```
# Plot cross-validation MSE against lambda
plot(cv_lasso)
```




```
# Save plot as PDF
pdf("Question10_LASSO_CV.pdf", width = 8, height = 6)
plot(cv_lasso)
dev.off()

## png
##   2
```

Step 4. Report `lambda.min`

`lambda.min` is the penalty value giving the smallest cross-validation mean squared error.

```
lambda_selected <- cv_lasso$lambda.min
lambda_selected

## [1] 0.5070394

# Minimum cross-validation MSE
min(cv_lasso$cvm)

## [1] 44.2725
```

Question 11: Fit LASSO and Identify Variables

Fit the LASSO model using the previously chosen `lambda.min`. Extract its coefficients and identify predictors with nonzero estimates; do not count the intercept.

Step 1. Fit LASSO using the selected λ

```
lasso_model <- glmnet(x = X, y = Y, alpha = 1,
  lambda = lambda_selected, family = "gaussian",
  standardize = TRUE)
# Display fitted model
print(lasso_model)
```

```
##
## Call:  glmnet(x = X, y = Y, family = "gaussian", alpha = 1, lambda
= lambda_selected,      standardize = TRUE)
##
##   Df  %Dev Lambda
## 1  8 61.94  0.507
```

Step 2. Extract the LASSO coefficient estimates

```
# Extract LASSO coefficients
lasso_coef <- coef(lasso_model)
print(lasso_coef)
```

```
## 16 x 1 sparse Matrix of class "dgCMatrix"
##                       s0
## (Intercept)  1.69029962
## X1           1.07944426
## X2           .
## X3           .
## X4          -1.92849719
## X5           0.41838736
## X6           0.06818692
## X7           0.89926976
## X8           .
## X9           .
## X10          6.62840784
## X11          .
## X12          .
## X13         -1.62217204
## X14         -1.16931949
## X15          .
```

```
# Convert sparse coefficients into a data frame
lasso_coef_df <- data.frame(
  Coefficient = rownames(lasso_coef),
  Estimate = as.numeric(lasso_coef)
)
print(lasso_coef_df)
```

```
##    Coefficient    Estimate
## 1  (Intercept)  1.69029962
## 2           X1  1.07944426
## 3           X2  0.00000000
## 4           X3  0.00000000
## 5           X4 -1.92849719
```

```
## 6           X5  0.41838736
## 7           X6  0.06818692
## 8           X7  0.89926976
## 9           X8  0.00000000
## 10          X9  0.00000000
## 11         X10  6.62840784
## 12         X11  0.00000000
## 13         X12  0.00000000
## 14         X13 -1.62217204
## 15         X14 -1.16931949
## 16         X15  0.00000000
```

Step 3. Identify selected predictors

```
# Exclude intercept
beta_lasso <- lasso_coef_df[-1, ]
# Identify predictors with nonzero coefficients
selected_predictors <- beta_lasso$Coefficient[
  beta_lasso$Estimate != 0
]
print(selected_predictors)
```

```
## [1] "X1"  "X4"  "X5"  "X6"  "X7"  "X10" "X13" "X14"
```

```
# Number of selected predictors
length(selected_predictors)
```

```
## [1] 8
```

Step 4. Create and save the coefficient table

Create a 15-row table for $\beta_1, \ldots, \beta_{15}$; a zero estimate means LASSO did not select that predictor. Save a CSV that students can copy into their report.

```
lasso_table <- data.frame(
  Coefficient = paste0("beta", 1:15),
  LASSO_Point_Estimate = as.numeric(
    lasso_coef[paste0("X", 1:15), 1]
  )
)
print(lasso_table)
```

```
##    Coefficient LASSO_Point_Estimate
## 1        beta1           1.07944426
## 2        beta2           0.00000000
## 3        beta3           0.00000000
## 4        beta4          -1.92849719
## 5        beta5           0.41838736
## 6        beta6           0.06818692
## 7        beta7           0.89926976
## 8        beta8           0.00000000
## 9        beta9           0.00000000
```

```
## 10      beta10           6.62840784
## 11      beta11           0.00000000
## 12      beta12           0.00000000
## 13      beta13          -1.62217204
## 14      beta14          -1.16931949
## 15      beta15           0.00000000

# Optional: save as CSV
write.csv(lasso_table, "Question11_LASSO_Coefficients.csv",
          row.names = FALSE)
```